Formalizing Hybrid Systems with Differential Logic in KeYmaeraX and Plaidypvs

Nicolas Crespo

Governor's School for Science and Technology

2023-2024

I have read this document and approve the content

_____

J. Tanner Slagel

NASA Langley Research Center

I have followed the Academic Integrity policy of the Governor's School for Science & Technology. I have neither given nor received unauthorized assistance in the preparation of this document.

_____

Nicolas Crespo

Abstract

In the development of safeguards for safety- and mission- critical systems, the verification of program effectiveness and correctness under any conditions is of the utmost importance. Differential Dynamic Logic (dL) provides a robust architecture for modeling such systems, incorporating differential equations to describe continuous dynamics, classical logic for complex behaviors, and various mathematical techniques to prove safety properties of a system. Implemented in Carnegie Mellon's KeYmaeraX and NASA's Prototype Verification System (PVS) through Plaidypvs (Properly Assured Implementation of Differential Dynamic Logic for Hybrid Program Verification and Specification), dL allows for the creation of formal proofs that are essential for confirming the reliability and functional safety of these systems. This research utilizes dL to develop and verify braking and collision avoidance systems for wildfire prevention drones with two examples. The first formalizes a scenario where a drone decelerates to a halt precisely at a predefined boundary. The second involves two drones navigating in a corridor, with a fail-safe mechanism to prevent collisions. These formalizations can be utilized to enhance the safety of automated systems, evaluate the effectiveness of KeYmaeraX and Plaidypvs, and contribute to the broader application and generalization of the Runtime Assurance (RTA) Framework.

Introduction

When creating safety-critical systems, ensuring the effectiveness of the underlying logic and behaviors is of the utmost importance. Although not made to deal with specifically dynamical systems, NASA has long been in the development of the Prototype Verification System (PVS), which includes a base of formally verified and concretely proven inner calculus and dozens of libraries (NASA PVS Library n.d.) covering topics from graph theory to polygons to probability algorithms. The library most relevant to reasoning about dynamical systems, however, is called dL, or Differential Dynamic Logic (Slagel, 2023).

The general syntax and most of the proof rules of dL have been specifically implemented in a way that makes it very similar to proving lemmas in PVS, but the fact that dL is an operational embedding inside PVS means most proof commands include a "$dl\ -$" prefix to signify that they have been created for and implemented for dL specifically (Slagel & Munoz, et. al., 2023). Furthermore, as stated by Dr. Tanner Slagel, "This embedding of dL is operational, meaning that it leverages the internal logic of PVS, resulting in a version of dL whose proof calculus is not only formally verified, but is also executable within PVS itself" (Slagel, 2023c).

Differential Dynamic Logic (dL) has a fundamental structure similar to classical logic, which facilitates the standardized writing and proving of theorems, regardless of their contents (Figure 1). However, dL separates itself from standard logic in its specified use in the formal analysis of differential equations, hence its name. As such, the term Hybrid Systems are described as such proofs that "contain continuously evolving and discretely evolving components" (Slagel, 2023a). This generally means that the proof is built with differential equations that describe continuous motion or change the value of variables continuously, along

with direct or instant assignment of variables, similar to how variables work in conventional programming.

The general structure of a proof written with dL in both KeYmaeraX and Plaidypvs, the implementation of dL in PVS, follows four basic parts (Figure 3). First, variables are defined, with specific syntax to describe whether they are to ever change in the program. This ensures constant variables are never tampered with or wrongfully changed. Second, some preconditions are specified, often being starting values or specified constraints for certain values. These values generally appear on the left side of a symbol of implication (Figure 5), as information to be assumed to be true. Third, the dynamical system of the program is described, albeit slightly differently in Plaidypvs and KeYmaeraX. Lastly, some postconditions are stated, or statements that are meant to be proven to be true once the program has finished executing (Slagel, 2023b).

Although both contain the fundamental concepts of Differential Dynamic Logic, implementing the proofs of a drone's braking and collision avoidance system in both KeYmaeraX and Plaidypvs can provide insight on the shortcomings and differing abilities of both frameworks.

The first example to be tested can be easily shown in a diagram (Figure 2, Figure 6), and describes a drone approaching a stationary barrier. A monitor on the drone knows its own position and current velocity, and until it is a certain distance away from the barrier, the drone's arbitrary, uncontrolled motion is allowed. However, once the monitor, which only checks the status of the drone's position and velocity at time intervals of some arbitrary value tau (τ) to retain a sense of realism, sees that the drone has passed some boundary a reversionary system takes over to ensure the drone does not collide with the barrier. This boundary is defined in terms of and is completely dependent on tau, the initial velocity and position, and the maximum

possible deceleration. This system is lastly wrapped in a *, or loop, signifying arbitrary repetition, meaning the system is allowed to repeat any number of times, and the postcondition must remain proven true regardless of the number of repetitions. The postcondition of this particular proof, although non-traditional, states that there is some run (∃) of the system that ends in the drone slowing to a stop at exactly the position of the barrier (Figure 7). The solving of this proof requires the use of various proof commands related to the feature of Hybrid Programs called a loop, which repeats a given system an arbitrary number of times and is signified by an asterisk (Figure 1). However, the flexibility of the existential quantifier (∃), which is used in the postcondition of this proof, allows the user to pull apart the loop however they please, practically negating its usage. By separating the proof into only the iterations of the loop that are needed, it can be easily communicated to the theorem prover that these iterations are the ones that will lead to the desired outcome. This is necessary because to solve a proof that states that there exists a run of this program that has some property, the exact details of that successful run must be provided to the theorem prover. In this case, derivations of the fundamental equations of motion were required.

The second example describes two drones, Drone A and Drone B, approaching each other in a corridor. Similar to the previous example, they are both controlled by some advanced controller outside of the proof's scope, and continuously perform some arbitrary motion under specified constraints. Once a monitor on Drone A sees that it is at a certain distance from Drone B, a reversionary system takes control of its motion, veers it out of the corridor without colliding with the arbitrarily moving Drone B, and redirects its course to its original pathing. The postcondition of this proof states that for all possible runs of this program, there is no outcome where Drone A and Drone B collide. Furthermore, since Drone A always veers to the same

direction during the possibility of a collision, implementing such behavior on all drones in an airspace can allow for the avoidance of all collisions, considering all behave as expected. The solving of this proof required the use of various novel calculus techniques, including the Differential Invariant and Differential Ghost rule (Figure 4), which, although adequately implemented in KeYmaeraX, have had minor problems in Plaidypvs. Research has been done around Differential Logic to produce these proof rules (Platzer & Tan, 2020), which are useful in the fact that they provide a way to leverage and reason about differential equations without solving for their solutions, which can be a computationally tedious and time consuming task.

The usefulness of these formalized systems is mainly in the area of wildfire prevention drones which NASA has been utilizing to prevent the spread of naturally and artificially started wildfires. As planes, drones and other vehicles crowd and are expected to increasingly crowd the airspace, effective braking and collision avoidance systems are crucial in ensuring their safety. A proper braking system is also useful especially for wildfire drones that must not leave a specified boundary and enter civilian areas. Formalizing and properly implementing systems for deceleration and collision avoidance can allow for the safe use of wildfire prevention drones or other unmanned vehicles without the possibility of crashes either into other drones or into other infrastructure.

Materials and Methods

Due to PVS's lack of support for ARM based CPUs, a Mac or Linux machine with a standard x86-64 CPU is needed to use Plaidypvs, and similar requirements for KeYmaeraX means it has very similar limitations. As a result, a Linux virtual machine on a M2 Macbook Pro and Windows Subsystem for Linux on a Windows computer were used when working with PVS or KeYmaeraX. While KeYmaeraX can be accessed through a browser, PVS was used alongside

its Visual Studio Code Extension in Visual Studio Code. Once a new version of PVS was finalized that supported CPUs with the ARM architecture, it was installed on the same M2 Macbook Pro and was used with the standard Emacs interface. Aside from type checking specifications written in PVS and the ability to run the Wolfram Alpha Engine within KeYmareaX, which are both negligible, the performance of these computers is not relevant.

Results

To complete the proof for the drone braking system in Example 1, a specification consisting of constants, changing variables, assumptions, and the hybrid system must first be created.

In this case, the only constant variables, though arbitrary, are bf and e, which represent the x location of the buffer line and the location of the wall the drone is approaching, respectively.

The changing variables are v, the velocity, x, the x position of the drone, a, the acceleration, and v_i, a variable that is used in the reversionary controller to store the initial velocity of the drone after it passes the buffer.

The assumptions to be made prior to the establishment of the hybrid system are as follows: $x \leq bf$, meaning the drone starts at or before the buffer, $bf < E$, meaning the buffer zone, or the distance between the buffer and the wall, is nonzero, $a > 0$, meaning the drone is accelerating towards the wall when the program starts, and $v > 0$, meaning the drone is moving forward towards the wall when the program starts.

Finally, the hybrid system contains an $++$ that represents non-deterministic choice with two different dynamics on either side of the $++$. The first describes the initial state of the system

by simply declaring that $x' = v$ and $v' = a$, with a domain restriction specified as $x \leq bf$. This ensures the advanced controller only is allowed to start when the drone has not yet reached the buffer zone. The second side of the $++$ describes the dynamics of the reversionary controller. Within this side of the $++$ a sequence of two steps is performed. First, with the expression $?x = bf;\ v_i := v$, the velocity of the drone is recorded once the drone reaches the buffer. This velocity, vi, is needed to calculate important expressions used later in the proof. Second, the expression $x' = v,\ v' = \frac{-v_i^2}{2(E-bf)}\ \&\ x \geq bf$ sets v', or the acceleration of the drone, to the expression $\frac{-v_i^2}{2(E-bf)}$. This expression was derived from the basic kinematic equations of motion using the initial velocity that was previously recorded and the distance between the buffer and the wall. The expression will make the drone decelerate towards the wall and have a velocity of zero once the drone reaches it. The domain restriction of these dynamics, $x \leq bf$, ensures that the reversionary controller is only allowed to start when the drone has passed the buffer. Lastly, the entire hybrid system is enclosed in a $*$, which represents arbitrary repetition. This ensures the system can be repeated as many times as needed.

Finally, the postconditions of the proof are described: that $x = E$, or the drone has reached the wall, and $v = 0$, or the drone has stopped. Since the system is surrounded with $<\ >$, it is established that the postconditions will use the existential quantified. In other words, this means the proof aims to show that these characteristics are true for some run of the system, as opposed to all runs of the system.

Results

The proof for the braking system specification relies heavily on the weakness of non-deterministic repetition when used alongside the existential quantifier. This allows the first steps of this proof to be simply using the $iterated()$ tactic to separate the first three runs of the program from the repetition. After separating each iteration with the $diamondOr()$ tactic and eventually removing the loop itself from the proof with the $orR()$ tactic, we can isolate only the iterations of the loop that are needed to fulfill the postconditions. After this, the $solve()$ tactic is used to solve and introduce a time variable to the differential equations of the hybrid system, and the $existsR()$ tactic is used to specify the exact time at which the postconditions are true relative to all other arbitrary characteristics of the system. Finally, two branches are created that each require basic simplification using assumptions made earlier on in the proof, and the proof can be completed with quantifier elimination. In other words, the theorem prover can complete the proof on its own because the final expressions are trivially true due to the correctness of the expression added with $existsR()$. Thus, the first specification was formally verified in KeYmaeraX. Recent developments and improvements to many of dL's proof rules, including $dl - solve()$ and $dl - diffinv()$, as well as essentially identical internal logic for proof rules in dL and tactics in KeYmaeraX, strongly suggest that the same fundamental logical step can be taken to also complete the proof with Plaidypvs and dL in PVS.

The increased complexity of the proof for Example 2 has slowed progress and not allowed a full specification to be written. However, some progress has been made in determining the conditions for the start of the reversionary controller. Using the basic kinematic equations of motion, it can be derived from the arbitrary variables the maximum distance the controlled drone must travel to safely escape the corridor. In this expression, $v_{max,x}$ is the constant velocity in the x direction, $v_{max,y}$ is the max velocity in the y direction, T , or $y = T$ is the top of the corridor,

and D is the maximum acceleration in the y direction. The expression states that drone A must be this far away from drone B for it to be possible for the reversionary controller to veer drone A to safety: $\Delta x = v_{constant,x}(\frac{T}{v_{max,y}} + \frac{3v_{max,y}}{2D})$. Taking this a step further by not assuming a constant x velocity, v_x, yields a slightly more complex equation that relies on a $min()$ and $max()$ function to separate time during acceleration and time during constant velocity:

$$\Delta x = [v_{initial,x} min((\frac{T}{3v_{max,y}} + \frac{v_{max,y}}{2D}),\ \frac{v_{max,x} - v_{initial,x}}{a})] + [v_{max,x} max(0,\ (\frac{T}{3v_{max,y}} + \frac{v_{max,y}}{2D}) - \frac{v_{max,x} - v_{initial,x}}{a})]$$

Although this expression is necessary in designing the specification of this proof, particularly the first assumptions to be made about the system, this work must be expanded upon for a full specification to be written and later proven. Both KeYmaeraX and Plaidypvs include significantly more tools for dealing with the universal quantifier ∀ than the existential quantifier ∃, which, once written, this proof will be more straightforward to complete. Although many of the existing examples for dL proofs in KeYmaeraX are much simpler, the same tactics would be used in this proof as in more trivial examples. Assuming the correctness and some degree of parity between the tactics in KeYmaeraX and proof rules in Plaidypvs, the proof will be formalizable with both platforms.

Discussion and Conclusion

Although not fully completed, recent developments in PVS's dL library suggests it has fully bridged the gap between Plaidypvs and KeYmaeraX's theorem proving capabilities. All of the proof rules used in the drone braking system example have been implemented in both systems meaning any proof could be recreated in both theorem provers. The second collision avoidance system has not yet reached a stage where a proof is possible, but the less complex

postcondition which uses the universal quantifier is much more common in Differential Dynamic Logic and therefore more thoroughly supported in both systems. Thus, the initial hypothesis that both specifications would be provable with KeYmaeraX and Plaidypvs is correct.

The use of KeYmaeraX and Plaidypvs for formal verification is somewhat more tedious due to its less obvious syntax (Figure 8) but using PVS is necessary for NASA to ensure correctness of these systems. These specifications rely on the correctness of the internal logic of the theorem-provers themselves to be valid, so using a more formally created system like PVS yields benefits. Given that both KeymaeraX and Plaidypvs are reliably correct, all initial assumptions must also not be overly general, and the specification itself must properly model the dynamical system for the proof to be valid. Since the program could theoretically control safety- and mission-critical systems, it is of the utmost importance that all levels of this program are fully formalized and correct. A mistake at any level could lead to catastrophic results for an automated vehicle that relies on such a system to manage its own safety.

For future implementations of differential logic, an added quantifier that considers time, called Temporal Differential Dynamic Logic, (White et. al., 2024), can be added to further the reasoning abilities of both Plaidypvs and KeYmaeraX. The more open-ended nature of Plaidypvs, being an embedding of dL in PVS, would much more easily allow for such an improvement, and development in dtL has already started. More specific to the previously stated examples, formalizing a generalization of the Runtime Assurance Framework (RTA) can also facilitate the future proof of similar systems containing an advanced system, monitor, and reversionary system. Overall, implementing such formalized logic in both manned and unmanned vehicles that require safety assurance can add a layer of safety to consumer based and industrial technologies.

Acknowledgements

I want to express my gratitude to Dr. Tanner Slagel, Dr. Lauren White, and Dr. Cesar Munoz for being at the forefront of development in PVS and creating Plaidypvs itself. Their guidance and attention have been invaluable in helping me learn to use PVS and become familiar with the structure of KeYmaeraX. Additionally, I extend my thanks to Mrs. Laura Vobrak, the coordinator of the Governor's School for Science and Technology's Mentorship Program, and all the other teachers who provided us with the necessary education and resources to pursue our research interests.

Literature Cited

Nasa. (n.d.). *NASA/pvslib: NASA PVS Library of formal developments*. GitHub. https://github.com/nasa/pvslib

Platzer, A., & Tan, Y. K. (2020). Differential equation invariance axiomatization. *Journal of the ACM*, *67*(1), 1–66. https://doi.org/10.1145/3380825

Slagel, J. T., Muñoz, C., Balachandran, S., Moscato, M., Dutle, A., Masci, P., & White, L. (2022). Towards an implementation of differential dynamic logic in PVS. *Proceedings of the 11th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*. https://doi.org/10.1145/3520313.3534661

Tanner Slagel. (2023, January 10). *Differential Dynamic logic in PVS*. GitHub. https://github.com/nasa/pvslib/blob/master/dL/README.md

White, L., Titolo, L., Slagel, J. T., & Muñoz, C. (2024). A temporal differential dynamic logic formal embedding. *Proceedings of the 13th ACM SIGPLAN International Conference on Certified Programs and Proofs*. https://doi.org/10.1145/3636501.3636943

Tanner Slagel. (2023). *Formally Verifying Runtime Assurance in Plaidypvs*. *YouTube*. Retrieved January 15, 2024, from https://www.youtube.com/watch?v=qtQIUJQRvug.

Figure 1: Hybrid program syntax in KeYmaeraX

Statements and effects of Hybrid Programs (HPs) and Hybrid Games (HGs).

HP	KeYmaera X	Operation	Effect
$x := e$	`x:=e;`	discrete assignment	assigns value of term e to variable x
$x := *$	`x:=*;`	nondet. assign	assigns any real value to variable x
$x' = f(x) \,\&\, Q$	`{x'=f(x)&Q}`	continuous evolution	evolve along differential equation $x' = f(x)$ within evolution domain Q for any duration
$?Q$	`?Q;`	test	check first-order formula Q at current state
$a; b$	`a; b`	seq. composition	HP b starts after HP a finishes
$a \cup b$	`a ++ b`	nondet. choice	choice between alternatives HP a or HP b
a^*	`{a}*`	nondet. repetition	repeats HP a n-times for any $n \in \mathbb{N}$
if(Q) a else b	`if(Q) {a} else {b}`	if-then-else	HP a runs if Q, otherwise b runs
a	`a;`	atomic program	run program symbol or subprogram a
a^d	`{a}^@`	dual game	HG: hybrid game duality operator
$a \cap b$	`a -- b`	demonic choice	HG: Demon's choice between HG a or b

Figure 2: Braking system diagram



Figure 3: KeYmaeraX proof structure

```
ArchiveEntry "LFCPS Textbook/07: Simple Discrete Loop"
Description "Example 7.2: Stronger Invariants for simple discrete loop".

ProgramVariables
  Real x, y;
End.

Problem
  x>=8 & 5>=y&y>=0
 ->
  [{x:=x+y; y:=x-2*y;}* @invariant(x>=y & y>=0)] x>=0
End.

Tactic "07: Simple Discrete Loop: stepwise"
  implyR(1) ; loop("x>=y&y>=0", 1) ; <(
    "Init": QE,
    "Post": QE,
    "Step": composeb(1) ; assignb(1.1) ; assignb(1) ; QE
)
End.

Tactic "07: Simple Discrete Loop: automatic"
  auto
End.

End.
```

Figure 4: dL proof rule cheat sheet

Hybrid program rules

$$\textbf{Mb}\ \frac{\vdash P \to Q}{\Gamma \vdash [\alpha]P \to [\alpha]Q, \Delta} \qquad \textbf{Gb}\ \frac{\vdash P}{\Gamma \vdash [\alpha]P, \Delta}$$

$$\textbf{Md}\ \frac{\vdash P \to Q}{\Gamma \vdash \langle\alpha\rangle P \to \langle\alpha\rangle Q, \Delta} \qquad \textbf{Gd}\ \frac{\vdash P}{\Gamma \vdash \langle\alpha\rangle P, \Delta}$$

$$\textbf{K}\ \frac{\Gamma \vdash [\alpha](P \to Q), \Delta}{\Gamma \vdash [\alpha]P \to [\alpha]Q, \Delta} \qquad \textbf{VRb}\ \frac{\Gamma \vdash P, \Delta}{\Gamma \vdash [\alpha]P, \Delta}\ \texttt{fresh?}(P)(\texttt{bv}(\alpha))$$

$$\textbf{loop}\ \frac{\Gamma \vdash J, \Delta \quad J \vdash [\alpha]J \quad J \vdash P}{\Gamma \vdash [\alpha^*]P, \Delta} \qquad \textbf{VRd}\ \frac{\Gamma \vdash P, \Delta}{\Gamma \vdash \langle\alpha\rangle P, \Delta}\ \texttt{fresh?}(P)(\texttt{bv}(\alpha))$$

$$\textbf{mbR}\ \frac{\Gamma \vdash [\alpha]Q, \Delta \quad Q \vdash P}{\Gamma \vdash [\alpha]P, \Delta} \qquad \textbf{mdR}\ \frac{\Gamma \vdash \langle\alpha\rangle Q, \Delta \quad Q \vdash P}{\Gamma \vdash \langle\alpha\rangle P, \Delta}$$

$$\textbf{mbL}\ \frac{\Gamma, [\alpha]Q \vdash \Delta \quad P \vdash Q}{\Gamma, [\alpha]P \vdash \Delta} \qquad \textbf{mdL}\ \frac{\Gamma, \langle\alpha\rangle Q \vdash \Delta \quad P \vdash Q}{\Gamma, \langle\alpha\rangle P \vdash \Delta}$$

$$\textbf{ghost}\ \frac{\Gamma \vdash [y := e]P, \Delta}{\Gamma \vdash P, \Delta}\ \texttt{fresh?}(y)$$

Differential equation rules

$$\textbf{dinit}\ \frac{\Gamma, Q \vdash [x' = f(x)\,\&\,Q]P, \Delta}{\Gamma \vdash [x' = f(x)\,\&\,Q]P, \Delta}$$

$$\textbf{dW}\ \frac{Q \vdash P}{\Gamma \vdash [x' = f(x)\,\&\,Q]P, \Delta}$$

$$\textbf{dI}\ \frac{Q \vdash [x' := f(x)](P)'}{P \vdash [x' = f(x)\,\&\,Q]P, \Delta}$$

$$\textbf{dC}\ \frac{\Gamma \vdash [x' = f(x)\,\&\,Q]C, \Delta \quad \Gamma \vdash [x' = f(x)\,\&\,(Q \wedge C)]P, \Delta}{P \vdash [x' = f(x)\,\&\,Q]P, \Delta}$$

$$\textbf{dG}\ \frac{\Gamma \vdash G,\ G \vdash P,\ \Gamma \vdash \exists y\,[x' = f(x),\ y' = a(x)\cdot y + b(x)\,\&\,Q]G, \Delta}{\Gamma \vdash [x' = f(x)\,\&\,Q]P, \Delta}$$

$$\textbf{dS}\ \frac{\Gamma \vdash [x' = f(x)\,\&\,Q]P}{\Gamma \vdash \forall t \geq 0\,(\forall 0 \leq s \leq t\,Q(y(s))) \to [x := y(t)]P}$$

Figure 5: A proof in Plaidypvs

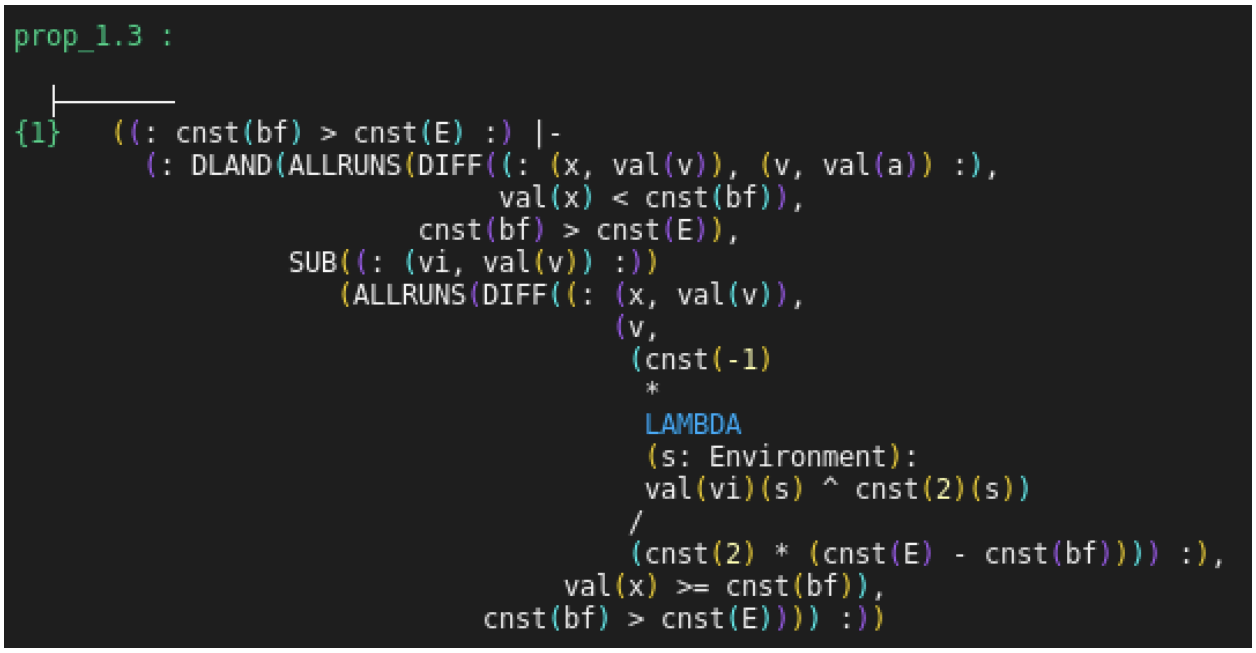


Figure 6: Diagram of the formalized drone's braking system

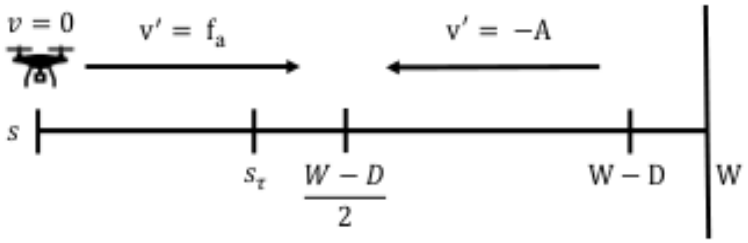


Figure 7: Example 1 braking system proof specification in KeYmaeraX

```
Theorem "Example 1"

Definitions
    Real bf;
    Real E;
End.

ProgramVariables
    Real v;
    Real x;
    Real a;
    Real vi;
End.

Problem
    (x<=bf) &
    (bf<E) &
    (0<a) &
    (v>0)
    ->
    <{{x'=v, v'=a & x<=bf} ++
    {{?x=bf; vi:=v;}; {x'=v, v'=-(vi^2)/(2*(E-bf)) & x>=bf}}}*
    >(x=E & v=0)
End.
```

Figure 8: Example 1 braking system proof the dL embedding of PVS (Plaidypvs)

```
%%
% @theory: Example1
% @author: nicolas
% @date: Wed, 08 Nov 2023 20:19:37 GMT
%%

typecheck-file | evaluate-in-pvsio | view-as-markdown
Example1: THEORY
  BEGIN

  IMPORTING dL@top

  % velocity
  v: dlVar = dlvar(1)

  % position
  x: dlVar = dlvar(2)

  % acceleration
  a: dlVar = dlvar(3)

  % buffer position (constant)
  bf: dlVar = dlvar(4)

  % end position (constant)
  E: dlVar = dlvar(5)

  % initial velocity (temp value, not constant)
  vi: dlVar = dlvar(6)

  prove | discharge-tccs | status-proofchain | show-prooflite
  prop_1 : LEMMA
    DLIMPLIES(val(x)<=cnst(bf) AND cnst(bf)>cnst(E) AND val(a)>cnst(0) AND val(v)>cnst(0),
      SOMERUNS(
        STAR(
          UNION(
            DIFF((: (x, val(v)), (v, val(a)) :), val(x)<=cnst(bf)),
            SEQ(
              SEQ(
                TEST(val(x)=cnst(bf)),
                ASSIGN((:(vi, val(v)):))),
              DIFF((: (x, val(v)), (v, (cnst(-1)*(val(vi)^cnst(2)))/(cnst(2)*(cnst(E)-cnst(bf)))) :),
            val(x)>=cnst(bf))))),
    DLAND(val(x)=cnst(E), val(v)=cnst(0))))

END Example1
```